Elastic wavefield reconstruction from boundary data in FDELMODC

Forward recording, equivalent (MPS) sources, time reversal and the backward kernel — all in FDELMODC's sign convention

Implementation notes

Generated from the code state of 18 Sep 2026

Contents

1 Goal and summary **1**

2 The fdelmodc elastic scheme and its sign convention **2**
2.1 The update equations as coded . . . 2
2.2 Continuous form and the stress sign . . . 2
2.3 Comparison with the paper's convention . . . 3

3 Equivalent sources on a closed boundary **3**

4 Discretisation on the staggered grid **3**
4.1 Contour points, normals, corners . . . 3
4.2 One interpolation table per field . . . 4
4.3 Amplitude: the $ds/\Delta z$ factor . . . 4

5 The forward (recording) run **5**

6 Reconstruction run: time reversal **5**
6.1 Setup common to both modes . . . 5
6.2 What time reversal does to the equations . . . 6
6.3 Default mode (`wfl_bwd=0`): forward kernel, Option S . . . 6
6.4 Backward kernel (`wfl_bwd=1`) . . . 6

7 Where things are in the code **8**

8 Results **8**

9 Known limitations and open items **9**

10 Reproducing and reverting **10**

1 Goal and summary

The goal is to implement the elastic wavefield-reconstruction scheme of Aaker et al. (2020, *GJI* **222**, 595–609) inside FDELMODC. In a *forward* run the five fields $v_x, v_z, \tau_{xx}, \tau_{zz}, \tau_{xz}$ are recorded on a closed rectangular contour $\Gamma = \partial\Omega$. In a second run, with *no real source*, equivalent sources built from the recorded data are injected on the same contour, in reversed time order, and the wavefield inside Ω is reconstructed.

Three things had to be right: (i) the injection must be the exact adjoint of the extraction (same interpolation weights, same staggered positions); (ii) the amplitudes of the equivalent sources must be consistent with the discrete scaling of the FDELMODC kernel; (iii) the *signs* of the sources and of the time-reversed data must be consistent with FDELMODC's equations.

The sign point is the subtle one and is the main content of this note:

- With the ordinary (forward-time) kernel, a time-reversed recording can reconstruct *either* the velocity *or* the stress with its true sign, never both. The first implementation (default, `wfl_bwd=0`) reconstructs velocity correctly and stress with flipped sign (measured amplitude ratio $k \approx -0.993$).
- Running the same update code with $\Delta t \to -\Delta t$ (the *backward kernel*, `wfl_bwd=1`) makes the time-reversed fields *native* solutions. All five fields then reconstruct with their true sign ($k \approx +0.993$ for all five, correlation 0.9997).

Section 8 gives the numbers and trace comparisons.

2 The fdelmodc elastic scheme and its sign convention

2.1 The update equations as coded

The elastic kernel (`elastic4.c`, fourth order) updates, with z increasing downward,

$$\begin{aligned} v_x &-= \texttt{rox}\,\big[D_x\tau_{xx} + D_z\tau_{xz}\big], & v_z &-= \texttt{roz}\,\big[D_x\tau_{xz} + D_z\tau_{zz}\big],\\ \tau_{xx} &-= \texttt{l2m}\,D_x v_x + \texttt{lam}\,D_z v_z, & \tau_{zz} &-= \texttt{l2m}\,D_z v_z + \texttt{lam}\,D_x v_x,\\ \tau_{xz} &-= \texttt{mul}\,\big[D_z v_x + D_x v_z\big], \end{aligned}$$

where D_x, D_z are the 4th-order staggered differences (coefficients $9/8$, $-1/24$) *without* the $1/\Delta x$ factor. That factor is built into the medium arrays in `readModel.c` with `fac`$= \Delta t/\Delta x$:

$$\texttt{rox} = \frac{\Delta t}{\Delta x\,\rho_x}, \quad \texttt{roz} = \frac{\Delta t}{\Delta x\,\rho_z}, \quad \texttt{l2m} = \frac{\Delta t}{\Delta x}(\lambda + 2\mu), \quad \texttt{lam} = \frac{\Delta t}{\Delta x}\lambda, \quad \texttt{mul} = \frac{\Delta t}{\Delta x}\mu. \tag{1}$$

Two facts used repeatedly below: (a) Δt enters the kernel *only* through these five arrays; (b) the same Δx (not a separate Δz) is used on both axes.

2.2 Continuous form and the stress sign

Dividing by Δt the coded update is the continuous system

$$\boxed{\rho\,\partial_t \mathbf{v} + \nabla\cdot\boldsymbol{\tau} = \mathbf{f}, \qquad \partial_t \boldsymbol{\tau} + \mathsf{C} : \nabla\mathbf{v} = \boldsymbol{s}} \tag{2}$$

with, componentwise,

$$\begin{aligned} &(\mathsf{C}:\nabla\mathbf{v})_{xx} = (\lambda{+}2\mu)\partial_x v_x + \lambda\partial_z v_z, \quad (\mathsf{C}:\nabla\mathbf{v})_{zz} = (\lambda{+}2\mu)\partial_z v_z + \lambda\partial_x v_x,\\ &(\mathsf{C}:\nabla\mathbf{v})_{xz} = \mu(\partial_z v_x + \partial_x v_z). \end{aligned}$$

Here $\mathbf{f}$ (force density) and $\boldsymbol{s}$ (deformation-rate source) are the two source terms. In the standard textbook form $\rho\partial_t\mathbf{v} = \nabla\cdot\boldsymbol{\sigma}^{\mathrm{std}}$, $\partial_t\boldsymbol{\sigma}^{\mathrm{std}} = \mathsf{C}:\nabla\mathbf{v}$ (tension positive). Comparing with (2),

$$\boxed{\boldsymbol{\tau}_{\mathrm{FDELMODC}} = -\boldsymbol{\sigma}^{\mathrm{std}}} \tag{3}$$

i.e. FDELMODC's stress is compression-positive (in the acoustic scheme $\tau_{zz} = p$). In the rest of this document I write $\boldsymbol{\tau}$ (or $\tau_{xx}, \tau_{zz}, \tau_{xz}$) for the FDELMODC stress fields and use the code's names `txx,tzz,txz`.

2.3 Comparison with the paper's convention

Aaker et al. use the Aki–Richards form $\rho\partial_t\mathbf{v} - \nabla\cdot\boldsymbol{\tau}^{\mathrm{AR}} = \mathbf{f}$, $\partial_t\boldsymbol{\tau}^{\mathrm{AR}} - \mathsf{C} : \nabla\mathbf{v} = -\mathsf{C} : \boldsymbol{h}$, with $\boldsymbol{\tau}^{\mathrm{AR}} = \boldsymbol{\sigma}^{\mathrm{std}} = -\boldsymbol{\tau}_{\text{FDELMODC}}$. Substituting, the paper's system becomes exactly (2) with $\mathbf{f} = \mathbf{f}^{\mathrm{paper}}$ and $\boldsymbol{s} = +\mathsf{C} : \boldsymbol{h}$. Their modified source functions are

$$\mathbf{f}^{\mathrm{paper}} = \rho \oint_\Gamma \delta(\boldsymbol{x} - \boldsymbol{x}_r)\, \frac{\boldsymbol{\tau}^{\mathrm{AR}}\cdot\mathbf{n}}{\rho(\boldsymbol{x}_r)}\, d\boldsymbol{x}_r \quad (\text{eq. } 23), \qquad \boldsymbol{h} = -\oint_\Gamma \delta(\boldsymbol{x} - \boldsymbol{x}_r)\, \mathbf{v}\otimes\mathbf{n}\, d\boldsymbol{x}_r \quad (\text{eq. } 24), \tag{4}$$

which in FDELMODC variables read (using $\boldsymbol{\tau}^{\mathrm{AR}} = -\boldsymbol{\tau}_{\text{FDELMODC}}$ and $\rho/\rho(\boldsymbol{x}_r) \approx 1$)

$$\mathbf{f} = -\boldsymbol{\tau}\cdot\mathbf{n}\,\delta_\Gamma, \qquad \boldsymbol{s} = -\mathsf{C} : (\mathbf{v}\otimes\mathbf{n})\,\delta_\Gamma. \tag{5}$$

I do not rely on the paper's signs, though: the next section derives (5) directly from (2), and the two agree.

3 Equivalent sources on a closed boundary

Let χ be the indicator of Ω ($\chi = 1$ inside, 0 outside) and $\mathbf{n}$ the *outward* unit normal, so that $\nabla\chi = -\mathbf{n}\,\delta_\Gamma$. Let $(\mathbf{v}, \boldsymbol{\tau})$ be a solution of (2) with a true source *outside* Ω (or none). The truncated field $(\chi\mathbf{v}, \chi\boldsymbol{\tau})$ obeys

$$\rho\partial_t(\chi\mathbf{v}) + \nabla\cdot(\chi\boldsymbol{\tau}) = \chi(\rho\partial_t\mathbf{v} + \nabla\cdot\boldsymbol{\tau}) + \boldsymbol{\tau}\cdot\nabla\chi = -\boldsymbol{\tau}\cdot\mathbf{n}\,\delta_\Gamma, \tag{6}$$

$$\partial_t(\chi\boldsymbol{\tau}) + \mathsf{C} : \nabla(\chi\mathbf{v}) = \mathsf{C} : (\mathbf{v}\otimes\nabla\chi) = -\mathsf{C} : (\mathbf{v}\otimes\mathbf{n})\,\delta_\Gamma. \tag{7}$$

So a wavefield that equals the true one inside Ω and is zero outside is generated *exactly* by the sources (5), using no medium knowledge except at Γ. This is the Huygens/representation-theorem statement behind Aaker et al. (2020). In components, with the recorded traction $\boldsymbol{T} = \boldsymbol{\tau}\cdot\mathbf{n}$,

$$T_x = \tau_{xx}n_x + \tau_{xz}n_z, \qquad T_z = \tau_{xz}n_x + \tau_{zz}n_z, \tag{8}$$

$$\mathsf{C} : (\mathbf{v}\otimes\mathbf{n})_{xx} = (\lambda{+}2\mu)v_xn_x + \lambda v_zn_z, \qquad \mathsf{C} : (\mathbf{v}\otimes\mathbf{n})_{zz} = \lambda v_xn_x + (\lambda{+}2\mu)v_zn_z, \tag{9}$$

$$\mathsf{C} : (\mathbf{v}\otimes\mathbf{n})_{xz} = \mu\,(v_xn_z + v_zn_x). \tag{10}$$

So the recorded τ feeds the *force* on v_x, v_z, and the recorded $\mathbf{v}$ feeds the *deformation-rate* sources on $\tau_{xx}, \tau_{zz}, \tau_{xz}$. That is the "MPS" (modified physical sources) structure.

4 Discretisation on the staggered grid

4.1 Contour points, normals, corners

`defineRecBoundary.c` builds the contour with spacing Δx on the top and bottom faces and Δz on the left and right faces. The top and bottom faces include the four corners; the left and right faces exclude them (so no duplicate points). For $\Omega = [500, 2500]^2$ m at 4 m this gives $2\cdot 501 + 2\cdot 499 = 2000$ contour points. Each point stores its position, its outward normal (Fig. 1) and an `is_corner` flag.

Corner weighting. At the four Cartesian corners the normal is discontinuous. Aaker et al. (Sec. 5.1) identify this as the largest error source and fix it with weight 1/2 on the corner points; the code does this by halving ds there.

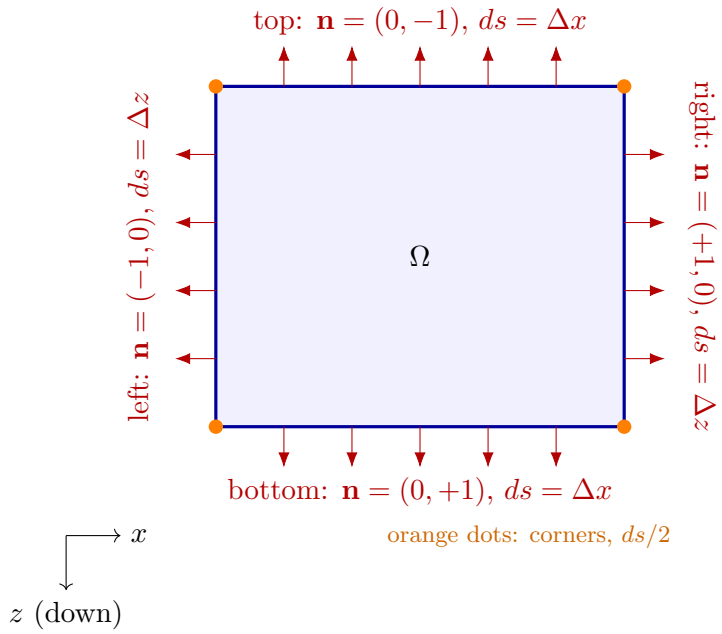


Figure 1: Contour Γ of $\Omega = [x_1, x_2] \times [z_1, z_2]$. Red arrows: outward normals. Orange dots: the four corners, which carry half weight.

4.2 One interpolation table per field

The five fields live at different staggered positions. From `readModel.c` (with the boundary padding indices `ioXx, ioZz, ...`) the physical position of field element `f[ix*naz+iz]` is

field	x offset	z offset
`vx`	$(\frac{1}{2} - \texttt{ioXx})\Delta x$	$-\texttt{ioXz}\,\Delta z$
`vz`	$-\texttt{ioZx}\,\Delta x$	$(\frac{1}{2} - \texttt{ioZz})\Delta z$
`txx,tzz`	$-\texttt{ioPx}\,\Delta x$	$-\texttt{ioPz}\,\Delta z$
`txz`	$(\frac{1}{2} - \texttt{ioTx})\Delta x$	$(\frac{1}{2} - \texttt{ioTz})\Delta z$

(index $\times$ spacing + offset; the padding terms encode the absorbing-layer shift). For each field, `bndinterpInit` (file `bndinterp.c`) computes for every contour point the base index and four bilinear weights $w_{00}, w_{01}, w_{10}, w_{11}$ and a `valid` flag for the 2×2 footprint.

The single primitive `bndinterp(inject, ...)` uses those weights in two directions:

- `inject=0` (extract): $d_i = \sum w\,$`field`$[\ldots]$;
- `inject=1` (inject): `field`$[\ldots]$ $+\!=\, w\, d_i$ (it never zeroes the live field).

Because both directions use the *same* table, injection is the exact adjoint of extraction by construction. This was checked by an adjoint (dot-product) test of the primitive.

4.3 Amplitude: the $ds/\Delta z$ factor

The equivalent sources (5) are line densities on Γ; the kernel wants a per-node increment. Take the force term. A discrete boundary point represents $F\,ds$ (force per length times length), spread over the cell area $\Delta x \Delta z$:

$$\Delta v = \frac{\Delta t}{\rho}\,\frac{F\,ds}{\Delta x\,\Delta z} = \underbrace{\frac{\Delta t}{\rho\,\Delta x}}_{\texttt{rox}}\,F\,\frac{ds}{\Delta z}.$$

The same steps for the stress sources give $\Delta\tau$ = `l2m`$(\dots)\,ds/\Delta z$ etc. So every increment is (medium array at the contour point)$\times ds/\Delta z\times$ (recorded data), and *not* $\Delta x\,ds\times\dots$ (an earlier version had this wrong: units of `rox`$\,\Delta x\,ds\,T$ are m^3/s, whereas `rox`$\,(ds/\Delta z)\,T$ is m/s). Δz is the uniform normaliser because `fac`$=\Delta t/\Delta x$ is used for both axes. With this scaling the reconstructed amplitude at the test point is within about 0.7% of the forward one (Sec. 8). The medium values (`rox`, `roz`, `l2m`, `lam`, `mul`) are sampled at the contour point through the field's own table, and $\rho(\boldsymbol{x})/\rho(\boldsymbol{x}_r)$ is taken as 1.

5 The forward (recording) run

Command-line: `wfl_recon=1 wfl_x1 wfl_x2 wfl_z1 wfl_z2 wfl_file=....` The run is an ordinary FDELMODC elastic run with the real source (in the demo a vertical force, `src_type=1 src_orient=1`, Ricker $f_p = 15$ Hz at $(1500, 1500)$). At the end of *every* time step it, in the OpenMP-master block of the main loop, after `elastic4()` has updated velocities and stresses, `storeBoundaryWav` does

```
bndinterp(0,0, vx , naz, tmp, rb.vx ) ; bs_vx [i*nt+it] = tmp[i];   /* likewise vz,txx,tzz,txz */
```

and the arrays `bs_*` (size $n_{\mathrm{contour}} \times n_t$, full resolution, not decimated by `dtrcv`) are written by `writeBoundaryWav` to `wfl_file`. The recorded values are the raw FDELMODC fields, i.e. the compression-positive $\boldsymbol{\tau}_{\mathrm{FDELMODC}}$ and the true $\mathbf{v}$. Nothing is negated at recording.

Timing note: the store is after the full step, so all five recorded fields are taken at the same loop index although $\mathbf{v}$ and $\boldsymbol{\tau}$ are staggered in time by a half step. This half-step mismatch is one plausible source of the 1–2 sample lags reported in Sec. 8; it has not been isolated by a dedicated test.

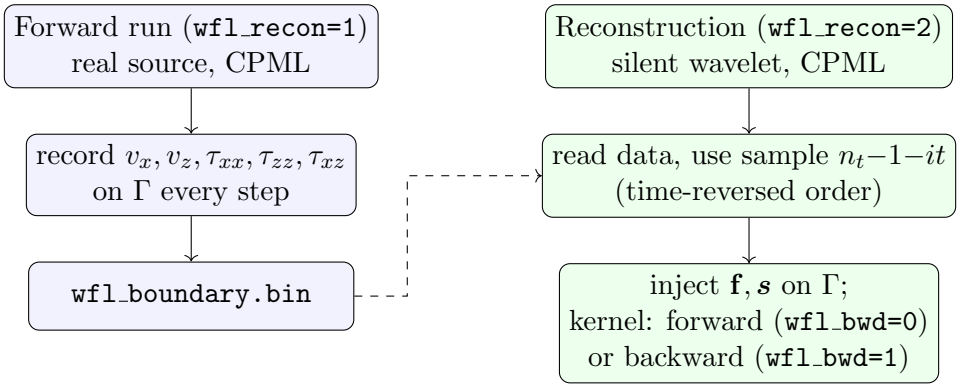


Figure 2: The two runs.

6 Reconstruction run: time reversal

6.1 Setup common to both modes

The reconstruction run has *no* real source. FDELMODC's command line has no "no source" mode (a `NULL` source file crashes `defineSource.c`), so the script passes a wavelet multiplied by exactly 0 (`sugain scale=0.0`; note `makewave scale=0` means "do not scale", not "zero"). The script verifies at runtime that this wavelet is exactly zero, and `applySource.c` skips zero-amplitude sources, so the run is driven *only* by the boundary injection.

At loop step it (loop time $s = it\,\Delta t$) the sample used is $n_t - 1 - it$, i.e. the recording is played backwards. In terms of the forward solution $(\mathbf{v}, \boldsymbol{\tau})(t)$ with $T = t_{\mathrm{mod}}$ define the time-reversed fields

$$\tilde{\mathbf{v}}(s) = \mathbf{v}(T - s), \qquad \tilde{\boldsymbol{\tau}}(s) = \boldsymbol{\tau}(T - s). \tag{11}$$

These are exactly what is read from the file at loop time s.

6.2 What time reversal does to the equations

Since $\partial_s \tilde{\mathbf{v}} = -(\partial_t \mathbf{v})(T-s)$ and likewise for $\tilde{\boldsymbol{\tau}}$, (2) (source-free) gives

$$\rho\, \partial_s \tilde{\mathbf{v}} = +\nabla\cdot \tilde{\boldsymbol{\tau}}, \qquad \partial_s \tilde{\boldsymbol{\tau}} = +\mathsf{C} : \nabla \tilde{\mathbf{v}}. \tag{12}$$

Both right-hand sides have the *opposite* sign to the forward system (2), which is the reason the forward kernel cannot reproduce $(\tilde{\mathbf{v}}, \tilde{\boldsymbol{\tau}})$ directly. Two other sign choices *do* solve the forward system:

$$\text{Option S:} \quad (\tilde{\mathbf{v}},\, -\tilde{\boldsymbol{\tau}}), \tag{13}$$

$$\text{Option V:} \quad (-\tilde{\mathbf{v}},\, +\tilde{\boldsymbol{\tau}}). \tag{14}$$

(Check Option S: $\rho\partial_s\tilde{\mathbf{v}} = \nabla\cdot\tilde{\boldsymbol{\tau}} = -\nabla\cdot(-\tilde{\boldsymbol{\tau}})$ and $\partial_s(-\tilde{\boldsymbol{\tau}}) = -\mathsf{C} : \nabla\tilde{\mathbf{v}}$, both of the form (2).) Exactly one of $\mathbf{v}, \boldsymbol{\tau}$ must therefore be negated with the forward kernel.

6.3 Default mode (`wfl_bwd=0`): forward kernel, Option S

This is the first implementation and is unchanged by the backward-kernel work. The truncation formulas (5) applied to the Option-S field $(\mathbf{v}_c, \boldsymbol{\tau}_c) = (\tilde{\mathbf{v}}, -\tilde{\boldsymbol{\tau}})$ give

$$\mathbf{f} = +\tilde{\boldsymbol{\tau}}\cdot\mathbf{n}\,\delta_\Gamma, \qquad \boldsymbol{s} = -\mathsf{C} : (\tilde{\mathbf{v}}\otimes\mathbf{n})\,\delta_\Gamma. \tag{15}$$

In the code, at load time `bs_vx, bs_vz` are multiplied by -1 (so the loaded v is $-\tilde{v}$) and `injectBoundary` then adds, with ds as above and the (positive) medium arrays,

```
tx = txx*nx + txz*nz;   tz = txz*nx + tzz*nz;          /* traction, raw tau  */
d_vx  = rox*(ds/dz)*tx;              d_vz  = roz*(ds/dz)*tz;
d_txx = (ds/dz)*(l2m*vxr*nx + lam*vzr*nz);             /* vxr = -vtilde     */
d_tzz = (ds/dz)*(lam*vxr*nx + l2m*vzr*nz);
d_txz = (ds/dz)*mul*(vxr*nz + vzr*nx);
```

which is $\mathbf{f} = +\tilde{\boldsymbol{\tau}}\cdot\mathbf{n}$ and $\boldsymbol{s} = -\mathsf{C} : (\tilde{\mathbf{v}}\otimes\mathbf{n})$ as required. The field this run produces is therefore $(\tilde{\mathbf{v}}, -\tilde{\boldsymbol{\tau}})$: the velocity has the true sign and **the stress is flipped**. This matches the measurement ($k_v \approx +0.993$, $k_\tau \approx -0.993$) and is not a bug in the source amplitude or in the interpolation: it is the sign that the forward kernel *must* produce for this choice (Option S). Choosing instead Option V would flip the velocity and leave the stress correct.

6.4 Backward kernel (`wfl_bwd=1`)

Idea. Because Δt enters the kernel only through `rox, roz, l2m, lam, mul` (1), negating those five arrays runs the very same update code with $\Delta t \to -\Delta t$:

$$\tilde{v} \mathrel{+}= |\texttt{rox}|\, D\tilde{\tau}, \quad \tilde{\tau} \mathrel{+}= |\texttt{l2m}|\, D\tilde{v} + \ldots$$

i.e. the discretisation of $\rho\partial_s\tilde{\mathbf{v}} = +\nabla\cdot\tilde{\boldsymbol{\tau}}$, $\partial_s\tilde{\boldsymbol{\tau}} = +\mathsf{C} : \nabla\tilde{\mathbf{v}}$, which is precisely the time-reversed system (12). So $(\tilde{\mathbf{v}}, \tilde{\boldsymbol{\tau}})$, both with their true signs, are *native* solutions and no field has to be negated. (Leapfrog is symmetric in Δt, so this is stable under the same CFL condition in the lossless interior.)

Sources. In the loop time s the equations with sources are $\rho\partial_s\tilde{\mathbf{v}} - \nabla\cdot\tilde{\boldsymbol{\tau}} = \mathbf{f}_b$, $\partial_s\tilde{\boldsymbol{\tau}} - \mathsf{C} : \nabla\tilde{\mathbf{v}} = \boldsymbol{s}_b$. Truncating with χ as in Sec. 3, and using $\nabla\chi = -\mathbf{n}\delta_\Gamma$,

$$\rho\partial_s(\chi\tilde{\mathbf{v}}) - \nabla\cdot(\chi\tilde{\boldsymbol{\tau}}) = -\tilde{\boldsymbol{\tau}}\cdot\nabla\chi = +\tilde{\boldsymbol{\tau}}\cdot\mathbf{n}\,\delta_\Gamma, \tag{16}$$

$$\partial_s(\chi\tilde{\boldsymbol{\tau}}) - \mathsf{C} : \nabla(\chi\tilde{\mathbf{v}}) = -\mathsf{C} : (\tilde{\mathbf{v}}\otimes\nabla\chi) = +\mathsf{C} : (\tilde{\mathbf{v}}\otimes\mathbf{n})\,\delta_\Gamma. \tag{17}$$

So in the backward kernel

$$\boxed{\mathbf{f}_b = +\tilde{\boldsymbol{\tau}}\cdot\mathbf{n}\,\delta_\Gamma, \qquad \boldsymbol{s}_b = +\mathsf{C} : (\tilde{\mathbf{v}}\otimes\mathbf{n})\,\delta_\Gamma} \tag{18}$$

with *unnegated* reversed data. Each increment is added with time step $+\Delta t$ in loop time, so the coefficients that multiply $ds/\Delta z$ are the *positive* magnitudes $|$`rox`$|$, etc. `injectBoundaryBwd.c` is a copy of `injectBoundary.c` that samples the (now negated) arrays through the same tables and multiplies by -1 to recover the positive values (not `fabsf`, since λ can be negative in general). The formulas are otherwise the ones of the listing above, with `vxr`$= +\tilde{v}_x$.

Consistency check with the default mode. Relative to the default mode the backward kernel flips the sign of the coupling (negated arrays) *and* of the injected velocity sources, which is what turns Option S into the native solution. The 1D analogue of this scheme was verified independently in a pure-python test: stress $k = +1.00$, velocity $k = +1.00$, leakage ≈ 0.03.

What the switch changes in the code. With `wfl_bwd=1` (only allowed together with `wfl_recon=2`), `fdelmodc.c` does three things, all behind `if (mod.wfl_bwd)`:

1. after `readModel`, negate `rox, roz, l2m, lam, mul` (whole arrays);
2. skip the negation of `bs_vx, bs_vz` at load;
3. call `injectBoundaryBwd` instead of `injectBoundary`.

With the flag unset (default) every code path is byte-for-byte the original one.

CPML. The CPML coefficients b, c, i_k are computed from `mod.dt, cp_min, dx, npml` and do not depend on the model arrays. In the PML cells the update is $\psi = b\psi + c\,d$; $d \leftarrow i_k d + \psi$; $\tau \mathrel{-}=$ `l2m`$(\dots)$, so negating `l2m` flips the coupling consistently while the memory recursion remains a decaying filter in loop time. In the test this worked (no instability, $k \approx +0.993$), but the CPML in reversed time is not proven stable in general; it has only been observed to behave.

7 Where things are in the code

file	role
`fdelmodc.h`	`modPar`: `wfl_recon, wfl_x1..wfl_z2, wfl_file, wfl_bwd`; include guard added
`getParameters.c`	parses the `wfl_*` parameters; `wfl_bwd` defaults to 0, error unless `wfl_recon=2`; elastic scheme only
`bndinterp.c/.h`	bilinear weight tables; extract/inject primitive (adjoint pair)
`bndsrc.h`	`bndSrcPar` struct, prototypes, derivation comments
`defineRecBoundary.c`	contour, normals, corner flags, five staggered tables
`storeBoundaryWav.c`	extraction each step (forward run)
`writeBoundaryWav.c`, `readBoundaryWav.c`	raw binary I/O with consistency checks
`injectBoundary.c`	default injection (forward kernel, Option S)
`injectBoundaryBwd.c`	backward-kernel injection, un-negated coefficients
`fdelmodc.c`	allocation, load, main-loop store/inject calls, the three `wfl_bwd` switches
`Makefile`	new sources; header-dependency line (stale objects caused a struct-layout crash once)
`demo/wfl_recon.scr`	forward run + default reconstruction + checks
`demo/wfl_recon_bwd.scr`	backward-kernel reconstruction, new output names
`backward_kernel_backup/`	original files, patch, `CHANGES.md` with revert commands

Main-loop sequence at step *it*. `elastic4()` (velocities, then stresses, CPML) → in the master block either `storeBoundaryWav(it)` (mode 1) or `injectBoundary[Bwd](..., nt-1-it, ...)` (mode 2) → receiver and snapshot output. The injection therefore happens after the step's update, so a velocity source is seen by the stress update only in the next step (a half-step timing approximation relative to injecting at each field's natural half step).

Demo parameters. $\Delta x = \Delta z = 4$ m, model 3100×3100 m, homogeneous $c_p = 2300$, $c_s = 1300$ m/s, $\rho = 2000$ kg/m^3; $\Delta t = 0.5$ ms, Ricker $f_p = 15$ Hz, $t_0 = 0.15$ s; $\Omega = [500, 2500]^2$ m; source at $(1500, 1500)$; independent receiver at $(2200, 800)$; `tmod`= 1.1 s (snapshots from 0.1 to 1.1 s each 0.05 s, so frame k of the reconstruction mirrors frame $N-1-k$ of the forward run); CPML with 20 cells on all four sides (`left=right=top=bottom=2`).

8 Results

The independent receiver at $(2200, 800)$ records the forward field (from the forward run) and the reconstructed field (from the reconstruction run, whose loop time runs backwards). For the comparison the reconstructed trace is reversed in time. Amplitude ratio k is the least-squares factor in recon $\approx k \cdot$ forward, at the best integer-sample lag within ± 6 samples.

	default (`wfl_bwd=0`)			backward kernel (`wfl_bwd=1`)		
field	lag	corr	k	lag	corr	k
`vx`	2	+0.9997	+0.9926	2	+0.9997	+0.9926
`vz`	2	+0.9997	+0.9925	2	+0.9997	+0.9925
`txx`	1	−0.9997	−0.9927	1	+0.9997	+0.9927
`tzz`	1	−0.9997	−0.9926	1	+0.9997	+0.9926
`txz`	1	−0.9997	−0.9923	1	+0.9997	+0.9923

The default run reproduces the velocity and returns the stress with the wrong sign; the backward kernel returns all five with $k \approx +0.993$. The lag is one to two samples (0.5–1 ms at

$\Delta t = 0.5$ ms). The amplitude is about 0.7% low in both modes; its cause (bilinear smoothing, the $ds/\Delta z$ point-source normalisation, or the small timing offset) has not been isolated.

Figure 3 overlays the forward trace and the time-reversed reconstruction (amplitudes normalised to the forward maximum), around the P-wave arrival, for v_z and τ_{xz}.

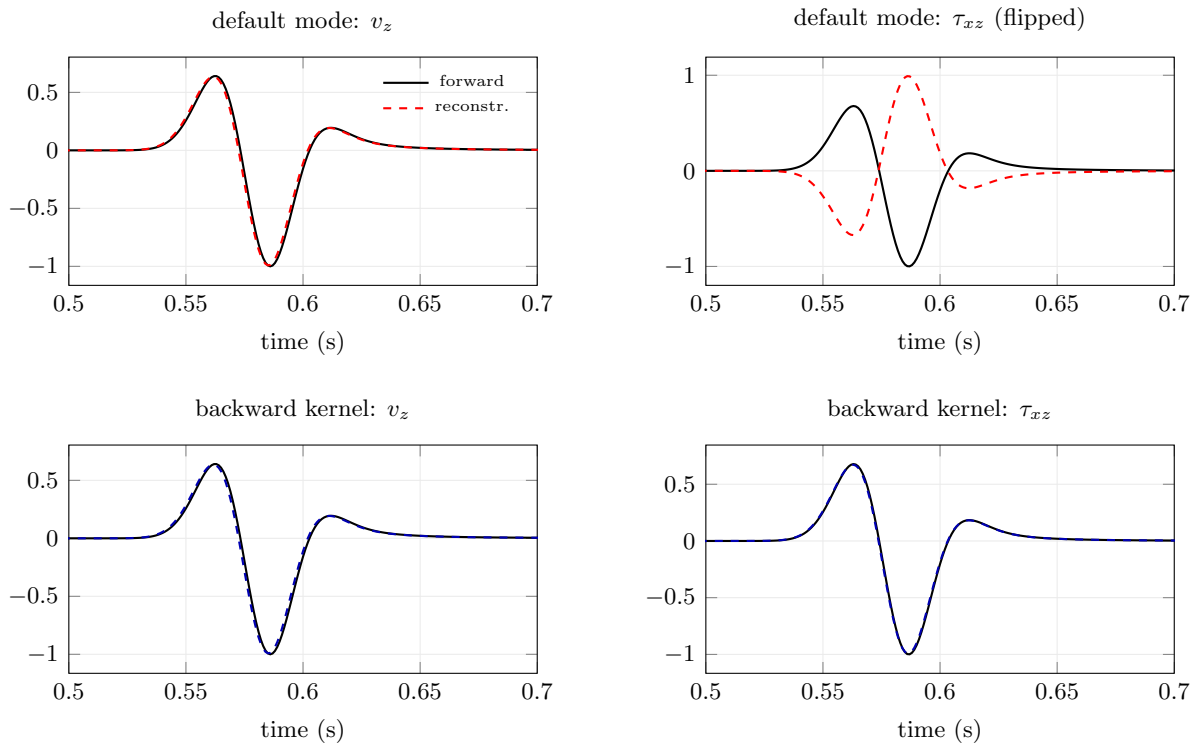


Figure 3: Point $(2200, 800)$: forward trace (black) and time-reversed reconstruction (dashed). Top: default mode, the stress is inverted. Bottom: backward kernel, both have the true sign.

Snapshots. `wfl_recon_bwd.scr` reverses the frame order of the reconstruction cube and forms `suop2 op=diff` against the forward cube for all five fields with *no* sign flips; inside Ω the difference should be small. Outside Ω the reconstruction is ideally zero, so the difference there just equals the forward field. I have not quantified the snapshot differences numerically in this document; the trace table above is the quantitative evidence.

9 Known limitations and open items

1. **Source inside Ω.** The demo source sits inside Ω. Aaker et al. (Sec. 3.4) note that boundary injection then reproduces the wavefield plus a spurious "homogeneous Green's function" part (after the wave has focused at the source position and expanded again), to be removed by also injecting the negated original source. That correction is *not* implemented. The compared window ($\approx$ 0.5–0.7 s forward time) corresponds to loop times before the focus at the source ($\approx T - t_0 \approx 0.95$ s), and by an arrival-time estimate the ghost cannot reach the test point within the run, so the comparison at that point is expected to be unaffected; this is an estimate, not a test.

2. **Initial condition.** The construction assumes the forward field inside Ω has vanished by t_{mod}. With $t_{\mathrm{mod}} = 1.1$ s the S-wave has not yet fully left Ω (S-travel time to the far corners is about 1.24 s), so the reconstruction start state is slightly incomplete. The effect has not been quantified.

3. **Half-step staggering.** Storage and injection happen after the full step, not at each field's own half step, giving the observed 1–2 sample lag (explanation plausible, not proven).

4. **Absorbing layers.** The former taper (`ntaper`, type 4) damps only the velocities and is not adequate; CPML (type 2) is used. CPML in reversed time works in the test but has no stability proof.

5. **Scope.** Elastic scheme only, homogeneous test, rectangular Ω, `iorder=4` exercised. Heterogeneity, other orders and viscoelastic media are untested.

10 Reproducing and reverting

```
cd fdelmodc && make                    # builds ../fdelmodc
cd demo
./wfl_recon.scr                        # forward run + default reconstruction (Option S)
./wfl_recon_bwd.scr                    # backward-kernel reconstruction (needs the first run)
```

Revert the backward-kernel work completely by restoring the five files from the folder `backward_kernel_backup/original/`, deleting `injectBoundaryBwd.c` and rebuilding (details in `backward_kernel_backup/CHANGES.md`); or simply do not pass `wfl_bwd=1`.